

tpcc_dbfs_vs_postgres_basic_comparison

May 21, 2026

1 TPCC Comparison: dbfs vs PostgreSQL

This notebook compares two BenchBase TPCC runs collected under the same nominal settings and grouped under a shared comparison label:

- comparison label: 20260503
- artifact selection: latest BenchBase artifact prefix under each engine directory
- dbfs result directory: `benchmarking/results/20260503/tpcc/dbfs`
- PostgreSQL result directory: `benchmarking/results/20260503/tpcc/postgres`

The notebook reads summary, metrics, params, samples, results, raw traces, per-procedure result windows, and optional query-trace CSVs from those directories to build side-by-side comparison tables and performance plots.

```
[1]: from pathlib import Path
import json
import re

import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import display

plt.style.use('ggplot')
pd.set_option('display.float_format', lambda value: f'{value:,.3f}')

comparison_root = Path(
    '/Users/yohasebe/Documents/dbfs/benchmarking/results/20260518-063146/tpcc')

# Query trace viewer (early stage - works without metrics.json)
#     dbfs_query_trace.csv

results_root = Path('/Users/yohasebe/Documents/dbfs/benchmarking/results')

#
latest_timestamp = None
latest_trace_path = None

for timestamp_dir in sorted(results_root.glob('**/tpcc/dbfs'), reverse=True):
    trace_path = timestamp_dir / 'dbfs_query_trace.csv'
```

```

if trace_path.exists():
    latest_timestamp = timestamp_dir.parent.parent.name
    latest_trace_path = trace_path
    break

if latest_trace_path:
    print(f"Loading query trace from: {latest_trace_path}")
    query_trace_df = pd.read_csv(latest_trace_path)

    # total_elapsed_us
    query_trace_sorted = query_trace_df.sort_values('total_elapsed_us',
                                                    ascending=False)

    # us ms
    display_df = query_trace_sorted.copy()
    display_df['total_elapsed_ms'] = display_df['total_elapsed_us'] / 1000
    display_df['mean_elapsed_ms'] = display_df['mean_elapsed_us'] / 1000
    display_df['max_elapsed_ms'] = display_df['max_elapsed_us'] / 1000

    display(display_df[[
        'kind', 'shape_key', 'executions', 'total_elapsed_ms',
        'mean_elapsed_ms', 'max_elapsed_ms', 'failures', 'last_error_type'
    ]])
else:
    print("No dbfs_query_trace.csv found in recent results")

```

Loading query trace from: /Users/yohasebe/Documents/dbfs/benchmarking/results/20260521-083833/tpcc/dbfs/dbfs_query_trace.csv

	kind	shape_key	executions	\
0	QUERY	SELECT COUNT(DISTINCT (S_I_ID)) AS STOCK_COUNT...	1	
1	QUERY	SELECT NO_O_ID FROM new_order WHERE NO_D_ID = ...	4	
2	QUERY	SELECT version();	1	
3	UPDATE	UPDATE order_line SET OL_DELIVERY_D = ? WHERE ...	3	
4	QUERY	SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_...	11	
5	QUERY	SELECT C_DISCOUNT, C_LAST, C_CREDIT FROM custo...	11	
6	QUERY	SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0...	111	
7	QUERY	SELECT I_PRICE, I_NAME , I_DATA FROM item WHER...	111	
8	UPDATE	INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W...	111	
9	QUERY	SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l...	3	
10	UPDATE	UPDATE stock SET S_QUANTITY = ? , S_YTD = S_YT...	111	
11	QUERY	SELECT OL_I_ID, OL_SUPPLY_W_ID, OL_QUANTITY, O...	2	
12	UPDATE	INSERT INTO oorder (O_ID, O_D_ID, O_W_ID, O_C_...	11	
13	UPDATE	UPDATE warehouse SET W_YTD = W_YTD + ? WHERE W...	15	
14	QUERY	SELECT W_STREET_1, W_STREET_2, W_CITY, W_STATE...	15	
15	UPDATE	UPDATE customer SET C_BALANCE = ?, C_YTD_PAYME...	15	
16	QUERY	SELECT D_NEXT_O_ID, D_TAX FROM district WHERE ...	11	
17	QUERY	SELECT D_STREET_1, D_STREET_2, D_CITY, D_STATE...	15	

18	UPDATE	UPDATE district SET D_YTD = D_YTD + ? WHERE D_...	15
19	QUERY	SELECT W_TAX FROM warehouse WHERE W_ID = ?	11
20	UPDATE	INSERT INTO history (H_C_D_ID, H_C_W_ID, H_C_I...	15
21	UPDATE	UPDATE district SET D_NEXT_O_ID = D_NEXT_O_ID ...	11
22	QUERY	SELECT O_ID, O_CARRIER_ID, O_ENTRY_D FROM oord...	2
23	QUERY	SELECT C_FIRST, C_MIDDLE, C_LAST, C_STREET_1, ...	6
24	QUERY	SELECT O_C_ID FROM oorder WHERE O_ID = ? AND O...	3
25	UPDATE	INSERT INTO new_order (NO_O_ID, NO_D_ID, NO_W...	11
26	UPDATE	UPDATE oorder SET O_CARRIER_ID = ? WHERE O_ID ...	3
27	UPDATE	DELETE FROM new_order WHERE NO_O_ID = ? AND NO...	3
28	QUERY	SELECT D_NEXT_O_ID FROM district WHERE D_W_ID ...	1

	total_elapsed_ms	mean_elapsed_ms	max_elapsed_ms	failures	\
0	30,123.274	30,123.274	30,123.274	1	
1	30,073.667	7,518.416	30,057.834	1	
2	5,010.481	5,010.481	5,010.481	1	
3	896.492	298.830	345.788	0	
4	541.144	49.194	75.248	0	
5	539.111	49.010	497.195	0	
6	505.911	4.557	43.284	0	
7	501.209	4.515	53.840	0	
8	485.835	4.376	107.423	0	
9	360.992	120.330	150.011	3	
10	342.839	3.088	25.572	0	
11	246.232	123.116	141.947	0	
12	111.511	10.137	84.762	0	
13	86.025	5.735	52.471	0	
14	56.113	3.740	12.653	0	
15	50.190	3.346	6.064	0	
16	46.225	4.202	11.395	0	
17	44.000	2.933	6.316	0	
18	41.336	2.755	6.698	0	
19	39.395	3.581	7.042	0	
20	31.196	2.079	5.501	0	
21	29.881	2.716	7.589	0	
22	28.273	14.136	17.734	0	
23	27.370	4.561	11.249	0	
24	21.364	7.121	10.645	0	
25	19.196	1.745	5.128	0	
26	7.644	2.548	3.405	0	
27	4.878	1.626	1.932	0	
28	2.944	2.944	2.944	0	

	last_error_type
0	SQLException
1	SQLException
2	SQLException
3	NaN

```

4           NaN
5           NaN
6           NaN
7           NaN
8           NaN
9   SQLException
10          NaN
11          NaN
12          NaN
13          NaN
14          NaN
15          NaN
16          NaN
17          NaN
18          NaN
19          NaN
20          NaN
21          NaN
22          NaN
23          NaN
24          NaN
25          NaN
26          NaN
27          NaN
28          NaN

```

```

[2]: run_configs = {
    'dbfs': {
        'label': 'tpcc_2026-05-18_06-31-46',
        'display_name': 'dbfs',
        'result_dir': comparison_root / 'dbfs',
    },
    'postgres': {
        'label': 'tpcc_2026-05-18_06-31-46',
        'display_name': 'PostgreSQL',
        'result_dir': comparison_root / 'postgres',
    },
}

configured_ratios = pd.Series(
    {
        'NewOrder': 45.0,
        'Payment': 43.0,
        'OrderStatus': 4.0,
        'Delivery': 4.0,
        'StockLevel': 4.0,
    },

```

```

    name='Configured Ratio (%)')

required_suffixes = {
    'summary': '.summary.json',
    'metrics': '.metrics.json',
    'params': '.params.json',
    'results': '.results.csv',
    'samples': '.samples.csv',
    'raw': '.raw.csv',
}

query_trace_filenames = {
    'dbfs': 'dbfs_query_trace.csv',
    'postgres': 'postgres_query_trace.csv',
}

procedure_suffix = '.results.'

```

```

[3]: dbfs_terminal_log = '''
[INFO ] 2026-05-03 04:09:32,266 [main]  com.oltpbenchmark.DBWorkload main -
↳=====
Benchmark:                                TPCC {com.oltpbenchmark.benchmarks.tpcc.
↳TPCCBenchmark}
Configuration:                            /benchbase/benchbase-config/dbfs_tpcc_docker.
↳xml
Type:                                     POSTGRES
Driver:                                   dev.yohaku.dbfs.jdbc.DbfsDriver
URL:                                     jdbc:dbfs://dbfs:25432/benchbase
Isolation:                               TRANSACTION_SERIALIZABLE
Batch Size:                              128
DDL Path:                                null
Loader Threads:                           2
Session Setup File:                       null
Scale Factor:                             1.0
Terminals:                               1
New Connection Per Txn:                   false
Reconnect on Connection Failure: true
[INFO ] 2026-05-03 04:11:29,210 [main]  com.oltpbenchmark.ThreadBench
↳runRateLimitedMultiPhase - PHASE START :: [Workload=TPCC] [Serial=false]
↳[Time=60] [WarmupTime=0] [Rate=10000.0] [Arrival=REGULAR] [Ratios=[45.0, 43.
↳0, 4.0, 4.0, 4.0]] [ActiveWorkers=1]
[INFO ] 2026-05-03 04:12:30,875 [main]  com.oltpbenchmark.DBWorkload
↳runWorkload - Rate limited reqs/s: Results(state=EXIT,
↳nanoSeconds=61382252527, measuredRequests=324) = 5.278398668368895 requests/
↳sec (throughput), 5.310981376198332 requests/sec (goodput)
Completed Transactions:

```

```

com.oltpbenchmark.benchmarks.tpcc.procedures.NewOrder/01
↳ [ 145]
↳*****
com.oltpbenchmark.benchmarks.tpcc.procedures.Payment/02
↳ [ 136]
↳*****
com.oltpbenchmark.benchmarks.tpcc.procedures.OrderStatus/03
↳ [ 18] *****
com.oltpbenchmark.benchmarks.tpcc.procedures.Delivery/04
↳ [ 14] *****
com.oltpbenchmark.benchmarks.tpcc.procedures.StockLevel/05
↳ [ 13] *****
Aborted Transactions:
<EMPTY>
Unexpected SQL Errors:
<EMPTY>
'''

postgres_terminal_log = '''
[INFO ] 2026-05-03 04:13:02,786 [main] com.oltpbenchmark.DBWorkload main -
↳=====
Benchmark: TPCC {com.oltpbenchmark.benchmarks.tpcc.
↳TPCCBenchmark}
Configuration: /benchbase/benchbase-config/
↳postgres_tpcc_docker.xml
Type: POSTGRES
Driver: org.postgresql.Driver
URL: jdbc:postgresql://postgres:5432/benchbase?
↳sslmode=disable&ApplicationName=tpcc&rewriteBatchedInserts=true
Isolation: TRANSACTION_SERIALIZABLE
Batch Size: 128
DDL Path: null
Loader Threads: 2
Session Setup File: null
Scale Factor: 1.0
Terminals: 1
New Connection Per Txn: false
Reconnect on Connection Failure: true
[INFO ] 2026-05-03 04:13:30,544 [main] com.oltpbenchmark.ThreadBench
↳runRateLimitedMultiPhase - PHASE START :: [Workload=TPCC] [Serial=false]
↳[Time=60] [WarmupTime=0] [Rate=10000.0] [Arrival=REGULAR] [Ratios=[45.0, 43.
↳0, 4.0, 4.0, 4.0]] [ActiveWorkers=1]
[INFO ] 2026-05-03 04:14:30,824 [main] com.oltpbenchmark.DBWorkload
↳runWorkload - Rate limited reqs/s: Results(state=EXIT,
↳nanoSeconds=60000046819, measuredRequests=24446) = 407.43301540656086
↳requests/sec (throughput), 405.2163504695948 requests/sec (goodput)

```

```

Completed Transactions:
com.oltpbenchmark.benchmarks.tpcc.procedures.NewOrder/01
↳ [11103]
↳*****
com.oltpbenchmark.benchmarks.tpcc.procedures.Payment/02
↳ [10298]
↳*****
com.oltpbenchmark.benchmarks.tpcc.procedures.OrderStatus/03
↳ [ 959] *****
com.oltpbenchmark.benchmarks.tpcc.procedures.Delivery/04
↳ [ 1007] *****
com.oltpbenchmark.benchmarks.tpcc.procedures.StockLevel/05
↳ [ 946] *****
Aborted Transactions:
com.oltpbenchmark.benchmarks.tpcc.procedures.NewOrder/01
↳ [ 134]
↳*****
Unexpected SQL Errors:
<EMPTY>
'''

run_configs['dbfs']['terminal_log'] = dbfs_terminal_log
run_configs['postgres']['terminal_log'] = postgres_terminal_log

field_patterns = {
    'workload': re.compile(r'Benchmark:\s+(\w+)'),
    'dbms_type': re.compile(r'Type:\s+(\w+)'),
    'driver': re.compile(r'Driver:\s+(.+)'),
    'url': re.compile(r'URL:\s+(.+)'),
    'isolation': re.compile(r'Isolation:\s+(\S+)'),
    'loader_threads': re.compile(r'Loader Threads:\s+(\d+)'),
    'scale_factor': re.compile(r'Scale Factor:\s+([\d.]+)'),
    'terminals': re.compile(r'Terminals:\s+(\d+)'),
}
phase_pattern = re.compile(
    r'PHASE START :: .*\[Time=(\d+)\] \[WarmupTime=(\d+)\] \[Rate=([\d.]+)\].
    ↳*\[Ratios=\[(\[^\]]+\)\]\]'
)
rate_pattern = re.compile(
    r'measuredRequests=(\d+)\) = ([\d.]+) requests/sec \((throughput\), ([\d.]+)
    ↳requests/sec \((goodput\)'
)

def parse_terminal_log(log_text):
    parsed = {}

```

```

for key, pattern in field_patterns.items():
    match = pattern.search(log_text)
    parsed[key] = match.group(1).strip() if match else None

phase_match = phase_pattern.search(log_text)
if phase_match:
    parsed['phase_time_seconds'] = int(phase_match.group(1))
    parsed['warmup_time_seconds'] = int(phase_match.group(2))
    parsed['target_rate'] = float(phase_match.group(3))
    parsed['configured_ratio_list'] = [
        float(value.strip()) for value in phase_match.group(4).split(',')
    ]

rate_match = rate_pattern.search(log_text)
if rate_match:
    parsed['terminal_measured_requests'] = int(rate_match.group(1))
    parsed['terminal_throughput'] = float(rate_match.group(2))
    parsed['terminal_goodput'] = float(rate_match.group(3))

return parsed

parsed_configs = pd.DataFrame([
    'engine': engine,
    **parse_terminal_log(config['terminal_log'])
] for engine, config in run_configs.items())
parsed_configs

```

```

[3]:      engine workload dbms_type                                driver \
0      dbfs      TPCC  POSTGRES  dev.yohaku.dbfs.jdbc.DbfsDriver
1 postgres      TPCC  POSTGRES                                org.postgresql.Driver

                                url \
0                                jdbc:dbfs://dbfs:25432/benchbase
1 jdbc:postgresql://postgres:5432/benchbase?sslmode=require

            isolation loader_threads scale_factor terminals \
0 TRANSACTION_SERIALIZABLE           2           1.0         1
1 TRANSACTION_SERIALIZABLE           2           1.0         1

phase_time_seconds  warmup_time_seconds  target_rate \
0                60                    0  10,000.000
1                60                    0  10,000.000

configured_ratio_list  terminal_measured_requests \
0 [45.0, 43.0, 4.0, 4.0, 4.0]                    324
1 [45.0, 43.0, 4.0, 4.0, 4.0]                    24446

```


	terminal_throughput	terminal_goodput
0	5.278	5.311
1	407.433	405.216

```
[4]: def find_latest_artifact_prefix(result_dir):
    summary_suffix = required_suffixes['summary']
    matches = sorted(result_dir.glob(f'*{summary_suffix}'))
    if not matches:
        raise FileNotFoundError(
            f'No artifact ending with {summary_suffix} under {result_dir}')
    latest_summary = matches[-1]
    return latest_summary.name[:-len(summary_suffix)]

def find_artifact(result_dir, run_label, suffix):
    artifact_path = result_dir / f'{run_label}{suffix}'
    if not artifact_path.exists():
        raise FileNotFoundError(
            f'No artifact ending with {suffix} for {run_label} under {result_dir}')
    return artifact_path

def load_optional_query_trace(engine, result_dir):
    trace_path = result_dir / query_trace_filenames[engine]
    if not trace_path.exists():
        return trace_path, None
    return trace_path, pd.read_csv(trace_path)

def load_run_artifacts(engine, config):
    result_dir = config['result_dir']
    detected_label = find_latest_artifact_prefix(result_dir)
    artifacts = {
        name: find_artifact(result_dir, detected_label, suffix)
        for name, suffix in required_suffixes.items()
    }
    procedure_paths = sorted(
        result_dir.glob(f'{detected_label}{procedure_suffix}*.csv'))
    query_trace_path, query_trace_df = load_optional_query_trace(
        engine, result_dir)

    with artifacts['summary'].open() as handle:
        summary = json.load(handle)
    with artifacts['metrics'].open() as handle:
```

```

        metrics = json.load(handle)
    with artifacts['params'].open() as handle:
        params = json.load(handle)

    results_df = pd.read_csv(artifacts['results'])
    samples_df = pd.read_csv(artifacts['samples'])
    raw_df = pd.read_csv(artifacts['raw'])
    procedure_dfs = {
        path.stem.split('.results.', 1)[1]: pd.read_csv(path)
        for path in procedure_paths
    }

    return {
        'label': detected_label,
        'artifacts': artifacts,
        'summary': summary,
        'metrics': metrics,
        'params': params,
        'results_df': results_df,
        'samples_df': samples_df,
        'raw_df': raw_df,
        'procedure_dfs': procedure_dfs,
        'query_trace_path': query_trace_path,
        'query_trace_df': query_trace_df,
    }

runs = {}
validation_rows = []
for engine, config in run_configs.items():
    run_data = load_run_artifacts(engine, config)
    runs[engine] = {**config, **run_data}
    validation_rows.extend([
        {
            'engine': engine,
            'artifact': 'results.csv',
            'rows': len(run_data['results_df']),
            'columns': ', '.join(run_data['results_df'].columns),
        },
        {
            'engine': engine,
            'artifact': 'samples.csv',
            'rows': len(run_data['samples_df']),
            'columns': ', '.join(run_data['samples_df'].columns),
        },
        {
            'engine': engine,

```

```

        'artifact': 'raw.csv',
        'rows': len(run_data['raw_df']),
        'columns': ', '.join(run_data['raw_df'].columns),
    },
    {
        'engine': engine,
        'artifact': 'procedure_results',
        'rows': len(run_data['procedure_dfs']),
        'columns': ', '.join(sorted(run_data['procedure_dfs'].keys())),
    },
    {
        'engine':
            engine,
        'artifact':
            query_trace_filenames[engine],
        'rows':
            0 if run_data['query_trace_df'] is None else len(
                run_data['query_trace_df']),
        'columns':
            'missing (optional)' if run_data['query_trace_df'] is None else
            ', '.join(run_data['query_trace_df'].columns),
    },
]
])

```

```

validation_df = pd.DataFrame(validation_rows)
validation_df

```

```

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[4], line 72
    68
    69 runs = {}
    70 validation_rows = []
    71 for engine, config in run_configs.items():
---> 72     run_data = load_run_artifacts(engine, config)
    73     runs[engine] = {**config, **run_data}
    74     validation_rows.extend([
    75         {

Cell In[4], line 29, in load_run_artifacts(engine, config)
    27 def load_run_artifacts(engine, config):
    28     result_dir = config['result_dir']
---> 29     detected_label = find_latest_artifact_prefix(result_dir)
    30     artifacts = {
    31         name: find_artifact(result_dir, detected_label, suffix)
    32         for name, suffix in required_suffixes.items()

```

```

Cell In[4], line 5, in find_latest_artifact_prefix(result_dir)
      1 def find_latest_artifact_prefix(result_dir):
      2     summary_suffix = required_suffixes['summary']
      3     matches = sorted(result_dir.glob(f'*{summary_suffix}'))
      4     if not matches:
----> 5         raise FileNotFoundError(
      6             f'No artifact ending with {summary_suffix} under
↳ {result_dir}')
      7     latest_summary = matches[-1]
      8     return latest_summary.name[:-len(summary_suffix)]

```

```

FileNotFoundError: No artifact ending with .summary.json under /Users/yohasebe/
↳ Documents/dbfs/benchmarking/results/20260518-063146/tpcc/dbfs

```

```

[ ]: summary_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    summary = run['summary']
    latency = summary['Latency Distribution']

    summary_rows.append({
        'Engine':
            run['display_name'],
        'Run Label':
            run['label'],
        'Measured Requests':
            summary['Measured Requests'],
        'Elapsed Seconds':
            summary['Elapsed Time (nanoseconds)'] / 1_000_000_000,
        'Throughput (req/s)':
            summary['Throughput (requests/second)'],
        'Goodput (req/s)':
            summary['Goodput (requests/second)'],
        'Requests per Minute':
            summary['Throughput (requests/second)'] * 60,
        'Throughput - Goodput Gap':
            summary['Throughput (requests/second)'] -
            summary['Goodput (requests/second)'],
        'Average Latency (ms)':
            latency['Average Latency (microseconds)'] / 1000,
        'Median Latency (ms)':
            latency['Median Latency (microseconds)'] / 1000,
        'P95 Latency (ms)':
            latency['95th Percentile Latency (microseconds)'] / 1000,
        'P99 Latency (ms)':
            latency['99th Percentile Latency (microseconds)'] / 1000,
    })

```

```

        'Max Latency (ms)':
        latency['Maximum Latency (microseconds)'] / 1000,
        'Terminal Throughput (req/s)':
        parsed.get('terminal_throughput'),
        'Terminal Goodput (req/s)':
        parsed.get('terminal_goodput'),
    })

```

```

summary_comparison_df = pd.DataFrame(summary_rows).set_index('Engine')
summary_comparison_df

```

```

[ ]: transaction_names = [
        'NewOrder', 'Payment', 'OrderStatus', 'Delivery', 'StockLevel'
    ]
transaction_pattern = re.compile(r'procedures\.(w+)/\d+s\[s*(\d+)\]')

def extract_histogram_counts(log_text, section_name):
    section_pattern = re.compile(
        rf'{section_name}:\n(.*)?(?:\n[w[w ]+:\nZ)',
        re.DOTALL,
    )
    match = section_pattern.search(log_text)
    if not match:
        return {name: 0 for name in transaction_names}

    block = match.group(1).strip()
    if '<EMPTY>' in block:
        return {name: 0 for name in transaction_names}

    counts = {name: 0 for name in transaction_names}
    for name, count in transaction_pattern.findall(block):
        counts[name] = int(count)
    return counts

mix_rows = []
for engine, run in runs.items():
    completed = extract_histogram_counts(run['terminal_log'],
                                         'Completed Transactions')
    aborted = extract_histogram_counts(run['terminal_log'],
                                       'Aborted Transactions')
    completed_total = sum(completed.values())
    aborted_total = sum(aborted.values())

    for name in transaction_names:
        mix_rows.append({

```

```

        'Engine':
run['display_name'],
        'Transaction':
name,
        'Completed':
completed[name],
        'Aborted':
aborted[name],
        'Observed Mix (%)': (completed[name] / completed_total *
                             100) if completed_total else 0.0,
        'Abort Rate (%)':
(aborted[name] / (completed[name] + aborted[name]) * 100) if
(completed[name] + aborted[name]) else 0.0,
    })

```

```

transaction_mix_df = pd.DataFrame(mix_rows)
transaction_mix_df

```

```

[ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    results_df = run['results_df']
    label = run['display_name']
    axes[0].plot(
        results_df['Time (seconds)'],
        results_df['Throughput (requests/second)'],
        marker='o',
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        marker='o',
        label=label,
    )

axes[0].axhline(
    summary_comparison_df.loc['dbfs', 'Throughput (req/s)'],
    linestyle='--',
    linewidth=1,
    label='dbfs overall avg',
)

axes[0].axhline(
    summary_comparison_df.loc['PostgreSQL', 'Throughput (req/s)'],
    linestyle=':',
    linewidth=1,
    label='PostgreSQL overall avg',
)

```

```

)
axes[0].set_title('5-second Throughput Trend')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('5-second P95 Latency Trend')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend()

fig.tight_layout()
plt.show()

```

```

[ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    samples_df = run['samples_df'].copy()
    results_df = run['results_df']
    label = run['display_name']
    samples_df['Throughput Rolling'] = samples_df[
        'Throughput (requests/second)'].rolling(5, min_periods=1).mean()

    axes[0].plot(
        samples_df['Time (seconds)'],
        samples_df['Throughput Rolling'],
        linewidth=2,
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['Median Latency (millisecond)'],
        marker='o',
        alpha=0.8,
        label=f'{label} p50',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        linestyle='--',
        alpha=0.8,
        label=f'{label} p95',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['99th Percentile Latency (millisecond)'],
        linestyle=':',

```

```

        alpha=0.8,
        label=f'{label} p99',
    )

axes[0].set_title('Per-second Throughput (5-second rolling avg)')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('Windowed Latency Percentiles')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend(ncol=2)

fig.tight_layout()
plt.show()

```

```

[ ]: procedure_rows = []
for engine, run in runs.items():
    for procedure_name, procedure_df in run['procedure_dfs'].items():
        procedure_rows.append({
            'Engine':
                run['display_name'],
            'Procedure':
                procedure_name,
            'Avg Throughput (req/s)':
                procedure_df['Throughput (requests/second)'].mean(),
            'Avg Latency (ms)':
                procedure_df['Average Latency (millisecond)'].mean(),
            'Avg P95 Latency (ms)':
                procedure_df['95th Percentile Latency (millisecond)'].mean(),
        })

procedure_comparison_df = pd.DataFrame(procedure_rows).sort_values(
    ['Procedure', 'Engine'])
display(procedure_comparison_df)

procedure_throughput = procedure_comparison_df.pivot(
    index='Procedure', columns='Engine', values='Avg Throughput (req/s)')
procedure_p95 = procedure_comparison_df.pivot(index='Procedure',
                                                columns='Engine',
                                                values='Avg P95 Latency (ms)')

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
procedure_throughput.plot(kind='bar', ax=axes[0])
axes[0].set_title('Average Throughput by Procedure')
axes[0].set_ylabel('req/s')

```



```

axes[0].tick_params(axis='x', rotation=30)

procedure_p95.plot(kind='bar', ax=axes[1])
axes[1].set_title('Average P95 Latency by Procedure')
axes[1].set_ylabel('Latency (ms)')
axes[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()

```

```

[ ]: mix_comparison_rows = []
for engine in transaction_mix_df['Engine'].unique():
    engine_mix = transaction_mix_df[transaction_mix_df['Engine'] ==
                                     engine].set_index('Transaction')
    for transaction_name, configured_ratio in configured_ratios.items():
        observed_ratio = engine_mix.loc[transaction_name, 'Observed Mix (%)']
        mix_comparison_rows.append({
            'Engine':
                engine,
            'Transaction':
                transaction_name,
            'Configured Ratio (%)':
                configured_ratio,
            'Observed Ratio (%)':
                observed_ratio,
            'Absolute Error (pp)':
                abs(observed_ratio - configured_ratio),
        })

mix_comparison_df = pd.DataFrame(mix_comparison_rows)
display(mix_comparison_df)

report_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    completed_total = transaction_mix_df.loc[transaction_mix_df['Engine'] ==
                                              run['display_name'],
                                              'Completed'].sum()

    aborted_total = transaction_mix_df.loc[
        transaction_mix_df['Engine'] == run['display_name'], 'Aborted'].sum()

    report_rows.append({
        'run_label':
            run['label'],
        'engine':
            run['display_name'],
        'dbms_type':

```

```

        parsed['dbms_type'],
        'workload':
        parsed['workload'],
        'scale_factor':
        float(parsed['scale_factor']),
        'terminals':
        int(parsed['terminals']),
        'duration_seconds':
        run['summary']['Elapsed Time (nanoseconds)] / 1_000_000_000,
        'measured_requests':
        run['summary']['Measured Requests'],
        'throughput_req_per_s':
        run['summary']['Throughput (requests/second)]',
        'goodput_req_per_s':
        run['summary']['Goodput (requests/second)]',
        'completed_transactions':
        int(completed_total),
        'aborted_transactions':
        int(aborted_total),
        'abort_percentage':
        (aborted_total / (completed_total + aborted_total) * 100) if
        (completed_total + aborted_total) else 0.0,
    })

```

```

report_df = pd.DataFrame(report_rows)
display(report_df)

```

```

[ ]: def summarize_dbfs_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['sample_sql'] = summary_df['sample_sql'].fillna(
        summary_df['shape_key'])
    summary_df['total_elapsed_ms'] = summary_df['total_elapsed_us'] / 1000.0
    summary_df['mean_elapsed_ms'] = summary_df['mean_elapsed_us'] / 1000.0
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_elapsed_us'] / 1000.0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                         ascending=[False, False])

    return summary_df[[
        'kind',
        'shape_key',
        'sample_sql',
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
    ]]

```

```

]]

def summarize_postgres_server_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['kind'] = 'QUERY'
    summary_df['shape_key'] = summary_df['query'].str.replace(
        r'\s+', ' ', regex=True).str.strip()
    summary_df['sample_sql'] = summary_df['query']
    summary_df['executions'] = summary_df['calls']
    summary_df['total_elapsed_ms'] = summary_df['total_exec_time']
    summary_df['mean_elapsed_ms'] = summary_df['mean_exec_time']
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_exec_time']
    summary_df['failures'] = 0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                         ascending=[False, False])

    return summary_df[[
        'kind',
        'shape_key',
        'sample_sql',
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
    ]]

]]

def summarize_postgres_query_trace(trace_df):
    if {
        'query', 'calls', 'total_exec_time', 'mean_exec_time',
        'max_exec_time'
    }.issubset(trace_df.columns):
        return summarize_postgres_server_query_trace(trace_df)
    return summarize_dbfs_query_trace(trace_df)

def escape_plot_label(text):
    return text.replace('$', r'\$')

def build_shared_trace_comparison(trace_summaries):
    if not {'dbfs', 'postgres'}.issubset(trace_summaries):
        return None

```

```

shared_df = trace_summaries['dbfs'].merge(
    trace_summaries['postgres'],
    on='shape_key',
    how='inner',
    suffixes=('_dbfs', '_postgres'),
)
if shared_df.empty:
    return shared_df

shared_df['sample_sql'] = shared_df['sample_sql_dbfs'].fillna(
    shared_df['sample_sql_postgres'])
shared_df['combined_total_elapsed_ms'] = (
    shared_df['total_elapsed_ms_dbfs'] +
    shared_df['total_elapsed_ms_postgres'])
shared_df['combined_max_elapsed_ms'] = shared_df[[
    'max_elapsed_ms_dbfs',
    'max_elapsed_ms_postgres',
]].max(axis=1)
shared_df['label'] = shared_df['sample_sql'].str.slice(0, 96)
shared_df['plot_label'] = shared_df['label'].map(escape_plot_label)
return shared_df.sort_values(
    ['combined_total_elapsed_ms', 'combined_max_elapsed_ms'],
    ascending=[False, False],
)

trace_summaries = {}
for engine, run in runs.items():
    trace_df = run['query_trace_df']
    if trace_df is None or trace_df.empty:
        print(
            f"{run['display_name']}: {query_trace_filenames[engine]} was not_
↳found."
        )
        continue

    if engine == 'dbfs':
        engine_summary_df = summarize_dbfs_query_trace(trace_df.copy())
    else:
        engine_summary_df = summarize_postgres_query_trace(trace_df.copy())

    engine_summary_df['Engine'] = run['display_name']
    engine_summary_df['time_share_pct'] = (
        engine_summary_df['total_elapsed_ms'] /
        engine_summary_df['total_elapsed_ms'].sum() * 100)
    engine_summary_df['label'] = engine_summary_df['sample_sql'].str.slice(
        0, 96)

```

```

engine_summary_df['plot_label'] = engine_summary_df['label'].map(
    escape_plot_label)
trace_summaries[engine] = engine_summary_df

for engine, summary_df in trace_summaries.items():
    display_name = runs[engine]['display_name']
    print(f'Top query shapes for {display_name}')
    display(summary_df[[
        'kind',
        'executions',
        'time_share_pct',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
        'sample_sql',
    ]].head(15))

shared_trace_df = build_shared_trace_comparison(trace_summaries)
if shared_trace_df is not None:
    overlap_summary_df = pd.DataFrame([
        'dbfs_shapes':
            len(trace_summaries['dbfs']),
        'postgres_shapes':
            len(trace_summaries['postgres']),
        'shared_shapes':
            len(shared_trace_df),
        'dbfs_only':
            len(trace_summaries['dbfs']) - len(shared_trace_df),
        'postgres_only':
            len(trace_summaries['postgres']) - len(shared_trace_df),
    ])
    print('Shared query shape overlap')
    display(overlap_summary_df)

    display(shared_trace_df[[
        'kind_dbfs',
        'executions_dbfs',
        'executions_postgres',
        'total_elapsed_ms_dbfs',
        'total_elapsed_ms_postgres',
        'max_elapsed_ms_dbfs',
        'max_elapsed_ms_postgres',
        'sample_sql',
    ]].head(15))

```

```

fig, axes = plt.subplots(1, 2, figsize=(16, 8))

plot_specs = [
    (
        'combined_total_elapsed_ms',
        'total_elapsed_ms',
        'Shared Query Shapes by Total Time',
    ),
    (
        'combined_max_elapsed_ms',
        'max_elapsed_ms',
        'Shared Query Shapes by Max Latency',
    ),
]

for axis, (combined_metric, metric, title) in zip(axes, plot_specs):
    top_df = shared_trace_df.nlargest(
        10, combined_metric).sort_values(combined_metric)
    positions = list(range(len(top_df)))
    axis.barh(
        [position - 0.2 for position in positions],
        top_df[f'{metric}_dbfs'],
        height=0.4,
        label='dbfs',
        alpha=0.8,
    )
    axis.barh(
        [position + 0.2 for position in positions],
        top_df[f'{metric}_postgres'],
        height=0.4,
        label='PostgreSQL',
        alpha=0.8,
    )
    axis.set_yticks(positions)
    axis.set_yticklabels(top_df['plot_label'])
    axis.set_title(title)
    axis.set_xlabel('Latency (ms)')

axes[0].legend()
fig.tight_layout()
plt.show()

```

1.1 Comparison Notes

Conclusion template:

- PostgreSQL shows a large throughput advantage over dbfs under the same TPCC configuration.

- dbfs completed the same workload shape without unexpected SQL errors, which means the current SQL support is sufficient for this TPCC slice.
- The main next step is performance-oriented investigation: loader behavior, per-procedure latency spikes, and the gap between dbfs and PostgreSQL throughput.
- If `dbfs_query_trace.csv` and `postgres_query_trace.csv` are present in the result directories, the notebook also ranks query shapes by total time, mean latency, and max latency to highlight bottlenecks on each engine.